

题目 0: 笔记

Lecture 03 · Data Types & Expression 数据类型与表达式

> Advanced Programming — 标识符、变量与赋值、内置数据类型、运算符与表达式、基本输入/输出

****大纲****: ① 标识符/变量/赋值 ② 内置数据类型 ③ 运算符与表达式 ④ 基本输入/输出

一、标识符 (Identifier)

给变量、函数、类、模块等对象起的名字。

****命名规则****

-  字母 (A-Z, a-z)、数字 (0-9)、下划线 `\_`；数字不能开头
-  空格及 `@` `#` `\$` `%` `!` 等特殊符号；不能是关键字 (`if`、`else`、`for`、`while`、`def`、`class` 等)

```
```python
 合法
my_var = 10
_name = "private"
variable1 = 5
user_name = "John"
_private = True
MyClass = "example"
```
```

```
```python
 非法
1variable = 10 # 不能数字开头
my-var = 20 # 不能用连字符
my var = 30 # 不能有空格
class = "test" # class 是关键字
@name = "John" # @ 不允许
```
```

****命名规范 (约定俗成, 不强制) ****

| | | |
|------------|---------------------------|----------------------|
| 规范 | 示例 | 用于 |
| --- | --- | --- |
| camelCase | `getText`、`setValue` | 函数 |
| snake_case | `user_name`、`total_count` | 变量、模块、文件 |
| PascalCase | `MyClass`、`UserProfile` | 类名 |
| UPPER_CASE | `PI`、`MAX_LIMIT` | 常量 |
| `_x` | 单前导下划线 | `_internal` 约定"私有" |

| `\_\_x` 双前导下划线 | `\_\_private` | 类中名称修饰 (name mangling) |

****注意****: 大小写敏感 (`myvar`/`MyVar`/`MYVAR` 是三个变量); 支持 Unicode (如中文变量名, 少用); `\_\_init\_\_` 等双下划线 (dunder) 方法有特殊含义。

二、变量与赋值 (Variable & Assignment)

- 变量是内存中存值的命名容器; 用 `=` 赋值, 本质是把名字****绑定 (引用) ****到对象上
- ****无需声明****: 类型由 Python 自动推断
- ****动态类型****: 类型可随时改变, 如 `x = 1` 之后可 `x = "hello"`
- ****大小写敏感****

三、内置数据类型

| 类型 | 取值 | 示例 |

| --- | --- | --- |

| Boolean 布尔 | `True` / `False` | `is\_student = True` |

| Integer 整型 | 整数 | `age = 25` |

| Float 浮点型 | 小数, 支持科学计数法 | `height = 168.6`、`2.99792458e8` |

| Complex 复数型 | 虚部用 ****j**** (非 i) | `c = 12.3 + 4.56j` |

| String 字符串 | 引号包裹的文本, 支持中文 | `name = "张子邦"` |

| NoneType 空型 | 仅一个值 `None` (类似其他语言的 null) | `supervisor = None` |

```
``python
is_student = True
age = 25
PI = 3.14159
LIGHT_SPEED = 2.99792458e8          # 光速
dist_sun_earth = 1.49597870700e11   # 日地距离
# print(dist_sun_earth / LIGHT_SPEED) → ≈ 499 秒 ≈ 8.3 分钟
c = 12.3 + 4.56j
major = "光电信息科学与工程"
supervisor = None
````
```

> **\*\*练习\*\***: 光从太阳到地球需多久? →  $1.49597870700e11 / 2.99792458e8 \approx 499 \text{ 秒} \approx 8.3 \text{ 分钟}$

## ## 四、字符串格式化 (String Formatting)

`%` 格式化 (printf 风格) 是最古老的字符串格式化方式, 用 `%` 运算符把值替换进字符串。

**\*\*常用格式说明符\*\***

| 说明符 | 含义 |  
| --- | --- |  
| `%s` | 字符串 |  
| `%d` | 整数 |  
| `%f` | 浮点数 |  
| `%x` | 十六进制（小写） |  
| `%o` | 八进制 |  
| `%%` | 字面量 `%` |

```
```python
name = "Alice"
print("Hello, %s!" % name)                # Hello, Alice!

name = "Bob"
age = 25
print("%s is %d years old." % (name, age)) # Bob is 25 years old.

pi = 3.14159265
print("Pi is approximately %.2f" % pi)    # Pi is approximately 3.14
print("Pi is approximately %10.3f" % pi)  # Pi is approximately      3.142
print("%5d" % 42)                        # "    42"   （右对齐）
print("%-5d" % 42)                       # "42   |"   （左对齐）
print("%05d" % 42)                       # "00042"   （零填充）
```
```

> 格式 `%10.3f`：总宽 10 位，小数点后 3 位；`%-5d` 负号表示左对齐；`%05d` 表示零填充。

## ## 五、数据类型转换（Data Type Conversion）

类型转换（type casting）指把值从一种类型变成另一种。

**\*\*隐式转换（Implicit, 自动）\*\***：Python 自动把"较小"类型转成"较大"类型，不丢失信息。

```
```python
num_int = 10
num_float = 2.5
result = num_int + num_float
print(result)          # 12.5
print(type(result))    # <class 'float'>
```
```

**\*\*显式转换（Explicit, 手动）\*\***：用内置函数主动转换。

|  |           |  |     |  |
|--|-----------|--|-----|--|
|  | 函数        |  | 转为  |  |
|  | ---       |  | --- |  |
|  | `int()`   |  | 整数  |  |
|  | `float()` |  | 浮点数 |  |
|  | `str()`   |  | 字符串 |  |
|  | `bool()`  |  | 布尔值 |  |
|  | `list()`  |  | 列表  |  |
|  | `tuple()` |  | 元组  |  |
|  | `set()`   |  | 集合  |  |
|  | `dict()`  |  | 字典  |  |

```

```python
x = "123"
x_int    = int(x)      # 123
x_float  = float(x)    # 123.0
x_str    = str(123)    # "123"
x_bool   = bool(x)     # True
x_list   = list(x)     # ['1', '2', '3']
```

```

> 仅在安全且不丢失信息时（主要是数值类型之间）Python 才自动转换；需要精确控制时用显式转换函数。

## ## 六、运算符（Operators）

运算符是对值和变量（操作数）执行操作的特殊符号或关键字。

### \*\*1. 算术运算符（Arithmetic）\*\*

|  |      |  |              |  |                     |                   |
|--|------|--|--------------|--|---------------------|-------------------|
|  | 运算符  |  | 说明           |  | 示例                  |                   |
|  | ---  |  | ---          |  | ---                 |                   |
|  | `+`  |  | 加            |  | <code>3 + 2</code>  | $\rightarrow 5$   |
|  | `-`  |  | 减            |  | <code>3 - 2</code>  | $\rightarrow 1$   |
|  | `*`  |  | 乘            |  | <code>3 * 2</code>  | $\rightarrow 6$   |
|  | `/`  |  | 除（结果为 float） |  | <code>7 / 2</code>  | $\rightarrow 3.5$ |
|  | `//` |  | 整除（向下取整）     |  | <code>7 // 2</code> | $\rightarrow 3$   |
|  | `%`  |  | 取余           |  | <code>7 % 2</code>  | $\rightarrow 1$   |
|  | `**` |  | 幂            |  | <code>3 ** 2</code> | $\rightarrow 9$   |

### \*\*2. 关系运算符（Relational）\*\*

|  |      |  |     |  |
|--|------|--|-----|--|
|  | 运算符  |  | 说明  |  |
|  | ---  |  | --- |  |
|  | `==` |  | 等于  |  |

| `!=` | 不等于 |  
| `>` `>=` | 大于 / 大于等于 |  
| `<` `<=` | 小于 / 小于等于 |

### **\*\*3. 逻辑运算符 (Logical) \*\***

| 运算符 | 说明 |  
| --- | --- |  
| `and` | 与 (两边都 True 才 True) |  
| `or` | 或 (一边 True 即 True) |  
| `not` | 非 (取反) |

### **\*\*4. 赋值运算符 (Assignment) \*\***

`=`、`+=`、`-=`、`\*=`、`/=`、`//=`、`%=`、`\*\*=`

### **\*\*5. 位运算符 (Bitwise) \*\***

`&` (与)、`|` (或)、`^` (异或)、`~` (取反)、`<<` (左移)、`>>` (右移)

### **\*\*6. 成员运算符 (Membership) \*\* & \*\*身份运算符 (Identity) \*\***

- 成员运算符: 检测值是否在序列中
- 身份运算符: 检测两个变量是否引用**\*\*内存中同一个对象\*\*** (不仅仅是值相等)

```
```python
# 成员运算符
print("a" in "apple")      # True
print("z" not in "apple")  # True
print(2 in {1, 2, 3})      # True

# 身份运算符
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b)              # True   (值相等)
print(a is b)              # False  (不同对象)
print(a is c)              # True   (同一对象)
```
```

## **## 七、基本输入/输出 (Basic Input / Output)**

Python 用内置函数 `input()` 读取用户输入、`print()` 显示输出。

```
```python
name = input("请输入你的名字: ") # input() 返回字符串
print("你好, " + name)
```
```

## ## 八、转义字符 (Escape Characters)

转义字符是反斜杠 ``\`` 加一个特殊字符, 用于在字符串中表示换行、制表符、引号等难以直接输入的字符。

| 转义                      | 名称           | 含义                          |
|-------------------------|--------------|-----------------------------|
| ---                     | ---          | ---                         |
| <code>\n</code>         | 换行           | 移到下一行                       |
| <code>\t</code>         | 制表符          | 水平制表                        |
| <code>\\</code>         | 反斜杠          | 字面量 <code>\\</code>         |
| <code>\'</code>         | 单引号          | 字面量 <code>'</code>          |
| <code>\"</code>         | 双引号          | 字面量 <code>"</code>          |
| <code>\r</code>         | 回车           | 光标移到行首                      |
| <code>\b</code>         | 退格           | 光标后退一格                      |
| <code>\f</code>         | 换页           | 分页符                         |
| <code>\v</code>         | 垂直制表         | 垂直制表符                       |
| <code>\a</code>         | 响铃           | 尝试蜂鸣                        |
| <code>\0</code>         | 空字符          | Null character              |
| <code>\ooo</code>       | 八进制          | 八进制值 <code>ooo</code> 对应的字符 |
| <code>\xhh</code>       | 十六进制         | 十六进制值 <code>hh</code> 对应的字符 |
| <code>\N{name}</code>   | Unicode 名称   | 按 Unicode 名称取字符             |
| <code>\uxxxx</code>     | Unicode 16 位 | 16 位十六进制值对应的字符              |
| <code>\Uxxxxxxxx</code> | Unicode 32 位 | 32 位十六进制值对应的字符              |

### 题目 1: 打印乘法口诀表(带格式控制)

程序图下:

# 9x9 乘法口诀表

# 要求:

- # 1. 使用转义字符 `\t` 对齐每一列, 使用 `\n` 换行
- # 2. 使用条件语句 `if`, 当乘积大于 20 时, 在数字后面加上 \* 标记
- # 3. 内层循环控制每行打印的列数 (第 `i` 行打印 `i` 列)

```
for i in range(1, 10): # 外层循环: 控制行数, i 从 1 到 9
 line = "" # 每一行的内容
 for j in range(1, i + 1): # 内层循环: 控制列数, 第 i 行打印 i 列
 product = i * j # 计算乘积
 if product > 20: # 条件判断: 乘积大于 20 时加 * 标记
 line += f"{j}x{i}={product}*\t"
 else:
 line += f"{j}x{i}={product}\t"
```

```

 line += f"{j}x{i}={product}\t"
 line += "\n" # 每行末尾用 \n 换行
 print(line, end="") # 打印这一行（不再额外换行）

```

运行结果如下：

```

PS E:\Advanced programming> & E:/miniconda/python.exe "e:/Advanced programming/作业/multiplication_table.py"
1x1=1
1x2=2 2x2=4
1x3=3 2x3=6 3x3=9
1x4=4 2x4=8 3x4=12 4x4=16
1x5=5 2x5=10 3x5=15 4x5=20 5x5=25*
1x6=6 2x6=12 3x6=18 4x6=24* 5x6=30* 6x6=36*
1x7=7 2x7=14 3x7=21* 4x7=28* 5x7=35* 6x7=42* 7x7=49*
1x8=8 2x8=16 3x8=24* 4x8=32* 5x8=40* 6x8=48* 7x8=56*8x8=64*
1x9=9 2x9=18 3x9=27* 4x9=36* 5x9=45* 6x9=54* 7x9=63*8x9=72* 9x9=81*
PS E:\Advanced programming>

```

## 题目 2：模拟密码输入系统

程序如下：

# 模拟登录程序

# 要求：

# 1. 用户最多有 3 次输入密码的机会（while 循环）

# 2. 正确密码为 "Py\\123"（实际字符串为 Py\123）

# 3. 每次输入后判断：正确则登录成功并退出；错误且有剩余次数则提示重试；3 次全错则锁定

# 4. 使用转义字符 \t 和 \n 美化输出

```

correct_password = "Py\\123" # 正确密码，实际字符串为 Py\123
max_attempts = 3 # 最多尝试次数
attempts = 0 # 已尝试次数

```

```
while attempts < max_attempts:
```

```
 password = input("请输入密码：")
```

```
 if password == correct_password:
```

```
 # 密码正确：登录成功并退出循环
```

```
 print("登录成功!\n 欢迎使用系统。")
```

```
 break
```

```
 # 密码错误：增加已尝试次数
```

```
 attempts += 1
```

```
 remaining = max_attempts - attempts
```

```
if remaining > 0:
```

```

 # 还有剩余次数：提示重试
 print(f'密码错误，还剩{remaining}次机会\t请重试!')
 else:
 # 3 次都错误：账号锁定
 print("账号已锁定!\n请联系管理员。")

```

运行结果如下：

```

PS E:\Advanced programming> & E:/miniconda/python.exe "e:/Advanced prog
ramming/作业/login_simulator.py"
请输入密码：Py\123
登录成功！
欢迎使用系统。
PS E:\Advanced programming> & E:/miniconda/python.exe "e:/Advanced prog
ramming/作业/login_simulator.py"
请输入密码：py\123
密码错误，还剩2次机会 请重试！
请输入密码：Py\123
密码错误，还剩1次机会 请重试！
请输入密码：PY\123
账号已锁定！
请联系管理员。
PS E:\Advanced programming>

```

### 题目 3：实时内存监控条

# 内存使用率实时监控程序

# 要求：

- # 1. 使用 psutil 库获取系统内存使用率
- # 2. 使用 time.sleep() 每 0.5 秒刷新一次
- # 3. 打印后不换行（使用 print 的 end 参数）
- # 4. 输出格式为进度条样式，例如：RAM usage: |=====| 50%
- # 5. 进度条总长度为 10 个字符，= 表示已用，空格表示剩余
- # 6. 使用 \r（回车符）回到行首实现原地刷新

```
import psutil
```

```
import time
```

```
while True:
```

```
 # 获取系统内存信息，percent 为内存使用率（百分比）
```

```
 mem = psutil.virtual_memory()
```

```
 percent = mem.percent
```

```
 # 计算进度条：已用部分用 =，剩余部分用空格
```

```
 # 将百分比 0~100 映射到 0~10 的进度条长度（如 32%→3 个=，89%→9 个=）
```

```
 used = round(percent / 10)
```

```
 bar = "=" * used + " " * (10 - used)
```

```
\r 回到行首原地刷新; end="" 不换行
print(f"RAM usage: |{bar}| {percent:.0f}%", end="\r")
```

```
time.sleep(0.5) # 每 0.5 秒刷新一次
```

运行结果如下:

```
PS E:\Advanced programming> & E:/miniconda/python.exe "e:/Advanced programming/作业/ram_usage_monitor.py"
RAM usage: |=====| 67%
```